Actividad Python - Videos

Guía de ejecución y funcionamiento

Cómo ejecutar los ejemplos

1. Instala Python 3. En Windows, durante la instalación conviene activar la opción para agregar Python al PATH.
2. Abre una terminal en la carpeta phyton/Videos.
3. Comprueba la instalación con python --version; en algunos equipos Windows también puedes usar py --version.
4. Ejecuta el archivo deseado, por ejemplo: python hello_word.py.

También puedes abrir cada archivo en VS Code, IDLE u otro editor de Python y ejecutarlo desde allí.

Ejemplos incluidos

01. hello_word.py	02. strings.py
03. variables.py	04. data_types.py

Las salidas de ejemplo se comprobaron ejecutando los archivos de esta carpeta.

1. Hola Mundo

Requerimiento de la actividad: Muestra en pantalla el mensaje Hola Mundo.

Archivo: hello_word.py

```
print("Hola Mundo")
```

Cómo funciona: La función print() recibe una cadena de texto y la escribe en la salida. Este ejemplo es un primer programa para comprobar que Python puede ejecutar un archivo.

Cómo ejecutar: python hello_word.py

Salida de referencia

```
Hola Mundo
```

Conceptos clave: Mostrar texto con print() y cadenas.

2. Variables y concatenación de cadenas

Requerimiento de la actividad: Guarda textos en variables, presenta una cadena multilínea y combina nombre y apellido.

Archivo: strings.py

```
    #Variables
text1 = "Hola"
text2 = "Holaaaaa"

text3 ="""
Hola
Me llamo Alejandro
Tengo 17 años
"""

print(text1)
print(text2)
print(text3)

#Concatenacion
name = "Alejandro"
last_name = "Quintero"
full_name = name + " " + last_name
print(full_name)
```

Cómo funciona: text1, text2 y text3 contienen cadenas; las comillas triples permiten escribir texto en varias líneas. El operador + concatena las cadenas name y last_name, con un espacio entre ambas.

Cómo ejecutar: python strings.py

Salida de referencia

```
Hola
Holaaaaa

Hola
Me llamo Alejandro
Tengo 17 años

Alejandro Quintero
```

Conceptos clave: Variables str, asignación, cadenas multilínea y concatenación.

3. Variables y f-strings

Requerimiento de la actividad: Define texto, nombres, edad y dinero; muestra un saludo personalizado.

Archivo: variables.py

```
saludo = "Hola Mundo"
print(saludo)   #La maquina imprime la variable saludo, que contiene el texto "Hola Mundo"
nombre = "Diego"
segundo_nombre = "Alejandro"
edad = 17
money = 20.5

    #seguir las recomendaciones de PEP8 para nombrar variables, es decir, usar minúsculas y guiones bajos para separar palabras.

print(f"¡Hola {nombre} {segundo_nombre}, tienes {edad} años y tienes ${money} en tu cuenta!")
```

Cómo funciona: Cada variable conserva un valor que puede reutilizarse. La f-string inserta nombre, segundo nombre, edad y dinero dentro de la frase mediante expresiones entre llaves.

Cómo ejecutar: python variables.py

Salida de referencia

```
Hola Mundo
¡Hola Diego Alejandro, tienes 17 años y tienes $20.5 en tu cuenta!
```

Conceptos clave: Variables, str, int, float y f-strings.

4. Tipos de datos

Requerimiento de la actividad: Declara ejemplos de tipos básicos y estructuras de datos; muestra el nombre y tipo de una variable.

Archivo: data_types.py

```
    #Tipos de datos
        #string (cadenas de texto)
nombre = "Alejandro"
apellido = "Quintero"
saludo = "Hola Mundo"
        #int (numeros simples)
edad = 25
numero = 10
        #float (numeros decimales)
altura = 1.71
temperatura = 27.3
        #bool/booleanos (valores de verdad)
boleeano = True
booleano_falso = False
    #Datos especiales
        #listas (Arrays)
lista = ["1", "2", "3"]
lista_abc = ["a", "b", "c"]
        #tuplas
tupla = (1, 2, 3)
        #set
set_variable = {1, 2, 3}
        #diccionario
diccionario = {
    'hello' : 'hola',
    'bye' : 'adios'
}
informacion_personal = {
    'name' : 'Alejandro',
    'age' : 17,
    'cats' : 2
}

print(nombre)
print(type(nombre))
```

Cómo funciona: El archivo presenta cadenas (str), enteros (int), decimales (float), booleanos (bool), listas, tuplas, conjuntos (set) y diccionarios (dict). type(nombre) informa el tipo del valor almacenado en nombre.

Cómo ejecutar: python data_types.py

Salida de referencia

```
Alejandro
<class 'str'>
```

Conceptos clave: Tipos básicos, colecciones, diccionarios, print() y type().